PERSONAL SKILL / 2026-10-27

SMART AGENT

Architecture, capabilities and management: a deep manual

A comprehensive guide to the renamed personal skill, selective modules, pinned sources and efficient verified work.

Evidence edition: 27 October 2026. Sources, models and runtime controls can change; verify current support when used.

26 modules / 16 source registrations / 485 indexed references

> This documents an instruction package and real management helpers. Referencing a product does not establish its runtime installation or validation.

Contents

1. Purpose, identity and the meaning of completion
2. Architecture: one coordinator, selective modules
3. Using the management system
4. Three.js, Atlas and Open3DLab
5. Superpowers: selected methods rather than compulsory gates
6. Priority reverse engineering: reverse-skill and REA
7. Agent Beacon and the boundaries of memory
8. Saving tokens intelligently and evaluating Headroom
9. Current models, new models and future freshness
10. PixelRAG and visual evidence
11. Cybersecurity tools: network, application and binary analysis
12. Cybersecurity tools: research, credentials, Wi-Fi and logs
13. Reverse proxy performance: monitoring and diagnosis
14. Preserved capabilities and end-to-end workflows
15. Maintenance, validation, publication and rollback

Appendices provide the module registry, commands, model observations, pinned sources and validation boundaries.

SMART AGENT / 01

Purpose, identity and the meaning of completion

SMART AGENT is the renamed and expanded Agent Engineering OS personal skill. Its display name is SMART AGENT, its installable folder and technical slug are smart-agent, and its invocation is `$smart-agent`. It preserves the existing engineering, creative, research, game, security and continuity guidance while adding a management layer and the sources requested in this update.

The skill is an operating reference for an already capable model. The current model remains responsible for judgment: understanding the outcome, selecting evidence, deciding what to implement, using available tools and verifying the result. Modules supply non-obvious guidance when relevant. They do not replace judgment with a compulsory sequence or require every task to pass through a team of agents.

Completion is defined through observable outcomes. A changed file establishes implementation, a passing check establishes the behavior that check actually exercised, and a verified remote revision establishes publication. None automatically establishes usability on a physical device, access to a cloud service, absence of security defects or future performance. This distinction is central to the skill's honesty.

The package does not upgrade Codex, train model weights, create credentials or add missing runtime tools merely through its instructions. The update creates a real local skill, executable management helpers and detailed documentation. The external integrations are deliberately identified as pinned guidance, optional runtime choices, installed prerequisites or unverified services. That makes it possible to use the useful parts without confusing documentation with execution.

SMART AGENT / 02

Architecture: one coordinator, selective modules

The entrypoint is SKILL.md. It contains operating principles, authorization and evidence boundaries, and links to specialized references. The module registry contains 26 entries. A registry entry has a stable ID, title, keywords, reference path, enablement default, related modules and a validation expectation. The registry is an index of useful guidance, not a collection of automatically running agents.

The management helper lives in scripts/manage.py. Source discovery lives in scripts/toolkit.py. The source manifest records 16 registrations and exact revisions, while the catalog contains 485 indexed skills, workflows and guides. An indexed guide is not equivalent to a separately installed program. Some source registrations describe a repository, while the separately installed Meowa skill is a distinct registration referring to that capability.

Three data layers remain separate: module metadata, upstream provenance and current project state. Module metadata explains what guidance exists. Provenance identifies what external revision was reviewed. Project state records what happened in the current assignment. Keeping these layers separate prevents a long historical report from becoming an instruction for every new task.

This architecture limits context growth through progressive disclosure. For an Arabic interface, the agent can load relevant UI and Atlas guidance; for a reverse proxy incident, it can load monitoring and log analysis; for a binary investigation, it can load reverse-engineering and the priority toolchain adapter. A simple stable question can still be answered directly. No registry match means the helper lacked a matching word, not that the model lacks the capability.

Using the management system

Run the helpers with Python 3.10 or newer, from the skill folder or using absolute paths. The runtime helpers use the standard library; repository/frontmatter validation separately uses PyYAML. In PowerShell, use the call operator `&` before a quoted Python executable. Existing dedicated runtimes can keep their historical directory names without becoming a second active skill.

`manage.py list` shows the registry and effective enablement. `show <id>` returns one entry. `recommend "query" --limit 5` produces a bounded lexical shortlist. `enable <id>` and `disable <id>` write local suggestion preferences outside the portable bundle. They do not install or remove programs and cannot turn off authorization, security or verification requirements. Explicit user instructions remain authoritative.

Preferences normally live under `${CODEX_HOME}/skill-state/smart-agent/modules.json`. A dedicated SMART_AGENT_STATE path isolates evaluation. `doctor` checks module reference integrity and source presence. `toolkit.py check` checks repository revisions, indexed file hashes and recorded notices. The historical Blender download is optional: if missing, the check reports that limitation as a warning rather than claiming to verify its checksum.

The fetch helper explicitly retrieves one recorded Git pin into an empty destination without running upstream setup. Existing targets are rejected so local work is preserved. An alternative destination requires a deliberate manifest-root update before index refresh. The ordinary offline refresh only rebuilds the index; it never pulls sources or proves unexpected edits trustworthy. Enablement and retrieval are tools for selecting context, not a substitute for reviewing what will execute.

SMART AGENT / 04

Three.js, Atlas and Open3DLab

Three.js provides the browser-3D source and official documentation requested for this skill. The reviewed source reports version 0.186.0, but consuming projects must follow their actual installed version and migration requirements. Guidance covers scene lifecycle, renderer choice, compatible core/addon imports, asset loading, resizing, cleanup, input and measured performance. WebGPU or TSL should be selected for a concrete reason with tested browser support rather than assumed to be a universal replacement.

Atlas contributes a React/TypeScript component catalog and Arabic RTL examples. At the reviewed revision, its UI package is a private workspace package distributed through a local pack workflow. It re-exports upstream implementations. The adapter therefore checks actual peer dependencies, providers, styles and client boundaries instead of assuming a public npm package can simply be installed. RTL/LTR, mixed-script text, keyboard focus, touch, dialogs and narrow screens require rendered tests.

Atlas's third-party notices document dependencies but do not establish a repository-wide license grant for all original content. No root-level grant was found in the reviewed snapshot. The skill links and studies the source; redistribution of its own code or fonts requires the applicable rights. The absence of a verified font redistribution license cannot be resolved by assuming a local font is free to bundle.

Open3DLab is included as a user-requested asset candidate. The automated website read encountered an Anubis challenge; no listing, asset download or per-asset license was verified. For an actual selection, record creator, asset page, version/date, license, attribution, commercial/derivative/redistribution conditions and suitability for the intended audience. Stage files, inspect formats and archive contents, preserve originals, optimize a copy and verify the imported asset in the actual engine. Inclusion is not blanket permission to reuse every asset.

SMART AGENT / 05

Superpowers: selected methods rather than compulsory gates

Superpowers is interpreted as obra/superpowers. Its named skills are available through the pinned catalog, including systematic debugging, focused test-driven work, planning, receiving review and verification before completion. The integration uses their practical questions and evidence habits when relevant instead of loading the entire methodology for every assignment.

For a defect, reproduce the behavior, inspect the failing boundary, identify a causal explanation, make a focused change and verify the affected path. For dependent work, keep a concise plan sufficient to manage dependencies and unresolved choices. For review feedback, compare the comment with the actual implementation and user consequence before changing code. These practices reduce avoidable rework without demanding a new ceremony for a simple edit.

Upstream instructions such as always invoking a skill, compulsory brainstorming, universal TDD, automatic subagent dispatch or branch-finishing routines do not become global rules in SMART AGENT. The actual instruction hierarchy, user request, environment and delegation permissions remain in force. Review-only work remains review-only, and a source prompt mentioning ship or merge does not authorize publication.

Verification is scaled to the consequence of error. A changed behavioral boundary may justify a focused regression test; a reversible wording edit may need metadata and link checks. A passing superficial test cannot replace a meaningful check, but repeating all checks without a new uncertainty also wastes time and context. The skill seeks appropriate evidence and clear delivery, with no claim that a methodology guarantees future intelligence or success.

SMART AGENT / 06

Priority reverse engineering: reverse-skill and REA

The last two repositories were explicitly marked important and are integrated as priority references for relevant reverse-engineering work. reverse-skill supplies scenario routing and case/evidence contracts across binaries, APKs, JavaScript, packet captures and security research. REA supplies an application/binary analysis CLI and MCP workflow with provider-specific capabilities. They complement the existing authorized reconstruction guide and Ghidra adapter; they are not invoked for ordinary source-repository architecture questions.

The adapter first determines the actual artifact, analysis question, authorization, platform, architecture, source availability and evidence needed. An offline sample can remain offline. Static inspection is preferred when it answers the question; runtime execution or target interaction requires the corresponding scope and a suitable environment. Record hashes, addresses/functions, commands, provider versions, observations, unknowns and the relationship from evidence to finding to conclusion.

Neither repository was bootstrapped as a runtime during this update. REA's setup can register MCP and install matching instructions, while native analysis needs suitable existing providers such as Ghidra, Hopper or IDA. Its compact advertised schemas and retained evidence references can reduce context overhead when the actual server supports them. Discover the real tool schema, preserve evidence IDs, and retrieve complete dossiers when a summary is insufficient. A CLI result does not establish MCP registration.

reverse-skill includes routers and toolchain bootstrap scripts with explicit side effects. The integration records their methodology but does not execute their setup, inject global rules, launch specialists or start targets. No upstream role map authorizes agents. A timeout also does not prove work stopped: inspect returned state and operation identity before retrying work that may have begun. Conclusions from decompilation and reconstruction remain interpretations to validate, not automatically original source or proof of ownership rights.

SMART AGENT / 07

Agent Beacon and the boundaries of memory

Beacon contributes ideas for cross-harness session evidence, replay and reusable lessons. A useful lesson records the condition in which it was observed, the source, date, evidence and limits. It remains revisable. A successful pattern in one repository does not become a universal instruction for unrelated projects or a permanent inferred preference.

The skill separates project checkpoints, user memory, session archives, derived knowledge and model weights. A project checkpoint can describe the current branch, changed files, completed checks and next action. A user-memory write follows the host's actual policy. Archiving a transcript does not automatically validate its contents, update model weights or authorize sending it elsewhere.

The reviewed Beacon setup may capture extensive agent history and connect to a cloud mode. This update therefore integrates references without installing a collector, enabling hooks, signing in or exporting histories. Any later runtime adoption should inspect local/cloud destinations, watched directories, redaction, retention, export/deletion and platform support within the user's applicable authorization.

For token efficiency, retrieve a concise relevant lesson rather than a full archive. Verify drift-prone facts against current files or services when the decision matters. Preserve provenance so an agent can distinguish a prior observation from current evidence. No personal-memory update was made as part of this skill maintenance; a package update and a memory write are different actions.

SMART AGENT / 08

Saving tokens intelligently and evaluating Headroom

Efficiency means completing the requested work with appropriate correctness and less total overhead. It includes input, output and reasoning tokens, retrieval and image costs, latency, duplicated work and failures that require another attempt. A shorter final response alone is not evidence of lower total usage. The update makes no measured savings claim because no comparable real workload telemetry was collected.

The default improvements are low-cost habits: read selected references, search before opening large files, request bounded results, keep exact errors and source locations while summarizing routine logs, batch independent reads, avoid unchanged polling and repeat passed checks only when changes or uncertainty justify it. A concise checkpoint reduces rediscovery after a long interruption. A worker is useful only when authorized and its benefit exceeds context duplication and handoff cost.

Headroom offers context compression and retrieval approaches, but source inclusion does not wrap Codex or insert a proxy. Evaluate a small isolated sample first if output volume warrants it. Preserve originals and retrieval pointers, then verify that compressed material retains negation, ordering, IDs, timestamps, units, exceptions, source citations and authorization boundaries. Retrieve the original whenever a consequential detail was omitted.

The reviewed source documents an older Codex wrapper regression involving temporary CODEX_HOME directories and chat-history visibility. Version-specific preservation and recovery deserve review before any wrapper experiment. The skill therefore starts with bounded retrieval, not automatic wrapping.

The token-report helper computes input-token arithmetic from supplied measured baseline/actual values. It rejects invalid baselines, preserves negative savings and marks whether same-task and quality assertions were supplied. It does not verify those assertions independently, model billing, cached-input discounts, output/reasoning use or latency. Synthetic test numbers validate the arithmetic only. For a real claim, compare equivalent tasks and acceptance checks with actual usage telemetry and report regressions as well as gains.

SMART AGENT / 09

Current models, new models and future freshness

The model inventory is deliberately not a permanent ranking. It combines a sanitized local catalog with a separately dated observation of actual execution surfaces. At the maintenance observation, Codex chat controls expose eight candidates, collaboration exposes five of those, and two hidden local-cache entries remain inventory-only. Exact effort support differs by surface and is recorded in the accompanying JSON, not inferred from a family name.

The refresh helper accepts newly discovered model IDs and effort values without requiring the code to recognize a fixed family. It copies selected fields instead of account identity or arbitrary catalog objects. Cache freshness, a source checksum and the age of the surface observation remain separate. Refreshing the cache cannot claim to refresh tool schemas; an old schema snapshot is a prompt to verify, not proof of current support.

Before a consequential route, inspect the exact live target control and relevant runtime/account constraints. Verify capabilities or price claims with current official evidence when necessary. A model present in one cache may be unavailable in a particular agent tool. A model description is not a measured benchmark. Subscription limits, API pricing, cached inputs and local inference do not share one interchangeable cost model.

Keep the user's coordinator and effort selection unless a supported change is requested or applicable instructions authorize it. Dispatching a worker does not switch the coordinator, and a catalog entry does not authorize delegation or a new chat. There is no global default change, automatic fallback ladder or background monitoring job installed here.

Future readiness means on-use discovery, honest dates, compatibility checks and deliberate maintenance. It cannot mean knowing unreleased models, guaranteeing access to future products or automatically learning while the application is closed. The skill remains adaptable because its decision process uses observed capabilities rather than a frozen list.

PixelRAG and visual evidence

PixelRAG addresses cases where text extraction loses the information needed for an answer: table structure, chart axes, spatial grouping, diagrams and other visual relationships. Its integration separates rendering, tile selection, embedding/indexing, serving and answer verification. A task needing one screenshot does not automatically need a full GPU-powered retrieval deployment.

Prefer ordinary searchable text when it already preserves the evidence cheaply and accurately. If visual retrieval is warranted, retrieve a bounded set of useful images and read the actual pixels. Record source URL/path, page or tile, capture date and surrounding context. Verify table headers, units, chart legends, scales and qualifications rather than relying on visual similarity alone. Look at neighboring tiles if a crop hides the meaning.

The source contains different dependency groups and deployment choices. Rendering, optional ML extras, local indexes and hosted search have distinct platform and data-flow requirements. A plugin for another harness does not prove compatibility with this Codex environment. This update installs no embedding model or screenshot pipeline and sends no private documents to a hosted search service.

For adoption, evaluate representative queries against expected evidence. Measure recall, answer correctness, difficult negatives, latency and image/token cost. An attractive screenshot is not proof that retrieval found the right source, and an upstream research result is not a local benchmark. The skill preserves the distinction between retrieval candidates and verified conclusions.

Cybersecurity tools: network, application and binary analysis

The cybersecurity module maps tools to questions rather than running them all. Nmap discovers scoped hosts, ports and services; record method, vantage, time and relevant script impact. A banner is evidence to validate, not final proof. Wireshark examines authorized captures and protocol behavior; preserve packet references, timestamps, capture loss and confidentiality, and remember that encrypted payloads require applicable decryption material.

Kali Linux is an assessment distribution, not one scanner and not a reason to replace the user's operating system. A suitable isolated VM and snapshot may be useful when an environment is actually needed. Burp Suite supports HTTP inspection and scoped web assessment. Distinguish manual replay, passive analysis and active scanning, account for edition differences, and avoid replaying state-changing requests casually.

Ghidra supports static disassembly, decompilation and cross-references. Record binary hash, architecture and relevant addresses; decompiled output is an interpretation requiring validation. Suricata contributes IDS/IPS and protocol evidence. Rule version, capture coverage, drops and false positives matter. Passive or offline inspection and inline blocking are different operational decisions. Match the installed stable version, because a documentation path named latest can point to development material.

Metasploit can validate a specific vulnerability hypothesis in an authorized lab or assessment. Inspect module effects and target compatibility, preserve a bounded objective and cleanup, and never treat a failed attempt as proof that the target is safe. OWASP ZAP supports passive/proxy analysis and scoped active or automated tests; define authentication, context and exclusions, then validate meaningful findings against actual behavior.

These integrations supply decision support and evidence expectations. No scanner was installed, no interception certificate trusted, no packet capture started and no target assessed in this skill update. A later assessment must use the actual agreed target and techniques; documentation does not expand its scope.

Cybersecurity tools: research, credentials, Wi-Fi and logs

OSINT Framework is a directory of public-source research paths and tools. Check the original source and its date, ambiguous identities, access conditions and relevance. It does not turn search matches into verified identity attribution, and it does not justify unnecessary personal-data aggregation. Directory links can have different registration and local-installation requirements.

Hashcat is useful for authorized offline password-hash evaluation. Confirm the permitted sample, correct hash format/mode, compute budget and evidence-handling rules. Controlled test samples are preferable when they answer the question. Results depend on tested assumptions and coverage; recovered secrets remain sensitive and should not be included in public reports or committed artifacts. This is distinct from attempting online account access.

Aircrack-ng supports authorized Wi-Fi assessment. Adapter/driver support, interface, SSID/BSSID, channel and physical test environment matter. Passive evidence and a lab reproduction may suffice. Injection or deauthentication can disrupt other users and requires the actual explicit assessment scope; it is not an incidental discovery step.

Splunk searches and correlates operational and security events when a real deployment is available. Inspect actual indexes, sourcetypes, fields, timezones, ingestion lag, retention and access. Use bounded queries and validate raw events before assuming a field extraction or a detection is correct. A query returning no events may reflect missing coverage rather than absence of an incident. Available platform features and apps depend on the actual edition and configuration.

Across all tools, the report should connect a reproducible observation to an interpretation and its limits. Record tool/version, target scope, timestamp, sanitized command or query, raw evidence location and confidence. Distinguish an alert, a suspicion, a reproduced defect and a verified fix. A clean scan or empty dashboard does not certify security, and operational monitoring cannot compensate for missing source data.

Reverse proxy performance: monitoring and diagnosis

The reverse-proxy module begins with the deployed path: client, DNS, CDN/WAF, TLS termination, proxy, upstream queue, application and dependencies. Define the affected time window, route and traffic cohort, a healthy comparison and the user-visible consequence. Measure request rate, p50/p95/p99 latency, errors, timeouts, retries, active connections, queues, upstream health and resource saturation with a consistent population.

NGINX access logs can expose request and upstream connect/header/response timings, status, bytes and request IDs when configured. These measurements can overlap; do not add them as independent phases. Retries and upstream groups may produce multiple values. Preserve that structure and distinguish missing values from zero, cache hits and error paths. Basic stub-status counters do not create a latency histogram.

HAProxy diagnosis uses the actual version's logging/timer semantics, frontend/backend/server statistics, queues, connection/response/total time, termination state and retries. Correlate proxy and application logs using a bounded request ID and consistent clocks. Protect credentials, cookies and private payloads. Restrict management endpoints and keep metrics labels bounded rather than exposing user IDs or unbounded raw URLs.

High connect time suggests a different next observation from a long upstream-header wait or slow transfer after headers. A 502/503/504 alone does not identify the failing component. Retries may amplify upstream overload, while cache changes may improve latency at the expense of correctness or tenant isolation. Do not hide a slow application by increasing every timeout. Preserve streaming, WebSockets and legitimate long requests when relevant.

For an authorized change, back up configuration, run the installed proxy's configuration check, use a reversible reload/deployment and compare the same traffic measures afterward. A syntax check does not prove operating success. Do not average per-instance p95 values to claim a fleet p95. Deliver the supported cause or remaining hypotheses, change, evidence, limits and rollback. This module does not install a monitoring service or establish an unmeasured speedup.

Preserved capabilities and end-to-end workflows

The previous domain guidance remains available. Software work preserves the repository and stack, implements the actual change and verifies relevant behavior. Research checks current or uncertain facts against appropriate sources. Security and reliability address trust boundaries, sensitive state, recovery and realistic failure paths. Computer/app work verifies the resulting state before repeating an interrupted action.

Game guidance requires a playable result, suitable controls, distinctive content, input/focus behavior, saves, packaging and performance evidence. Meowa and Blender references support asset families and engine import, while an installed CLI is kept separate from authenticated generation or a connected Blender application. Creative production preserves the brief and supports deliberate visual identity, motion, sound and repeatable exports when available.

Example: for an Arabic RTL dashboard, inspect the existing product, choose relevant UI/Atlas examples, keep compatible primitives, implement, then check both directions, focus, touch and narrow layouts. Example: for a supplied binary, record its hash and question, use the reverse adapter and actual provider capabilities, retain evidence and validate the important interpretation. Example: for a slow proxy route, correlate a narrow log window with upstream evidence, make an authorized focused change and compare like-for-like metrics.

For resumption, reconcile the checkpoint with the actual checkout, branch/revision, changed files, results and remaining authorization. A checkpoint is useful context, not proof the files still match it. For creative or game delivery, a build is one piece of evidence; it does not establish a successful launch, usable input or correct exports. The skill keeps these practical boundaries while choosing only the workflows necessary for the current objective.

Maintenance, validation, publication and rollback

Maintenance starts from the installed bundle and the real repository rather than assuming they match. Preserve a backup and Git history, migrate the folder/slug/UI metadata coherently, check references and examples, then validate the installed copy and publishable copy separately. Keep one active personal coordinator. Existing source/runtime directories may keep old names because relocating working dependencies adds risk without improving the skill.

The repository validation checks metadata, reference discoverability, invocation coherence and local links. Fifteen automated tests exercise structural failures and meaningful helper behavior: isolated enablement, bounded recommendations, unknown future model IDs, exclusion of account identity, honest missing measurements, negative savings, invalid telemetry, path traversal, changed content and optional attachment reporting. The explicit fetch helper was also exercised against an exact Superpowers revision in a separate temporary destination without running upstream setup.

Integrity checks compare the indexed source files with their recorded hashes and repository revisions, and preserve notice checksums. They do not assess all upstream software for security or establish live account connections. The optional historical Blender package is currently absent and is disclosed. Open3DLab access and per-asset rights remain unverified. Runtime execution of the new external products and real token-savings benchmarks are outside this update's completed validation.

For future source maintenance, inspect new commits and licenses in a separate checkout, adapt only useful changes, update pins deliberately and rebuild the offline index. For models, refresh cache and schema observations independently. For publication, verify the actual remote revision and files; a tool attempt or local commit is insufficient. A changed README does not install the skill elsewhere.

Rollback can restore the previous skill backup or Git revision, or revert an individual module/pin/helper change. Preferences remain outside the bundle. Keep sensitive logs, credentials, raw sessions and personal records out of public outputs. The lasting result is a maintainable skill with explicit evidence boundaries, not a promise that every future tool, model or project is already supported and tested.

Appendix: module registry

ID	Purpose
browser-motion-and-audio	Browser motion and procedural audio
capability-orchestration	Capability-first orchestration
computer-work	Computer, app, and environment work
continuity	Continuity Guard
creative-production	Creative production workflow
game-assets-and-blender	Game assets, Meowa and Blender
game-development	Game development workflow and engine guides
game-testing-and-release	Game testing, performance and release
integrated-toolkit	Integrated toolkit
visual-retrieval	Session knowledge and visual retrieval
model-routing	Current models and future changes
motion-video-production	Motion and video production
project-team	Adaptive project collaboration
research	Evidence-driven research
reverse-engineering	Artifact and behavior reconstruction
security-reliability-infrastructure	Security, privacy, reliability, and infrastructure engineering
skill-management	SMART AGENT module management
software-engineering	Software and AI-agent engineering
token-efficiency	Token efficiency without losing evidence
visual-assets-and-performance	Asset preparation and delivery
visual-motion-and-review	Motion design and visual review
visual-web	Autonomous Visual Web
web-3d	Three.js and external 3D assets
cybersecurity-tools	Cybersecurity tools and evidence
reverse-proxy-observability	Reverse proxy performance monitoring and troubleshooting
reverse-toolchains	Priority reverse-skill and REA toolchains

Appendix: practical commands

Run from the skill folder. Disable and enable are separate commands; replace disable with enable as needed. These are technical command names, not a language requirement.

Purpose	Command
Inventory	`python scripts/manage.py list`
Recommend	`python scripts/manage.py recommend "reverse rea binary" --limit 5`
Proxy guidance	`python scripts/manage.py recommend "nginx upstream latency"`
Inspect module	`python scripts/manage.py show reverse-toolchains`
Disable suggestions	`python scripts/manage.py disable visual-retrieval`
Enable suggestions	`python scripts/manage.py enable visual-retrieval`
Local diagnostic	`python scripts/manage.py doctor`
Search source	`python scripts/toolkit.py search "debugging" --source superpowers --limit 5`
Exact document	`python scripts/toolkit.py read "rea:.agents/skills/reverse-engineer-anything/SKILL.md"`
Integrity	`python scripts/toolkit.py check`
Acquire chosen pin	`python scripts/fetch_sources.py --source superpowers`
Offline reindex	`python scripts/toolkit.py refresh`
Model cache snapshot	`python scripts/manage.py refresh-models --surfaces assets/surface-models.json`
Measured arithmetic	`python scripts/manage.py token-report measurements.json`

Appendix: models by execution surface

Observed 27 October 2026. This is neither a ranking nor permanent access. Hidden cache entries gpt-reserve and codex-auto-review remain inventory-only and are not exposed by these controls.

ID	Chat controls	Collaboration
gpt-6.1-sol	low, medium, high, xhigh, max, ultra	low, medium, high, xhigh, max, ultra
gpt-6-astra	low, medium, high, xhigh, max, ultra	low, medium, high, xhigh, max, ultra
gpt-6.1-terra	low, medium, high, xhigh, max, ultra	low, medium, high, xhigh, max, ultra
gpt-6-sol	low, medium, high, xhigh, max, ultra	low, medium, high, xhigh, max, ultra
gpt-6-luna	low, medium, high, xhigh, max	low, medium, high, xhigh, max
gpt-5.6-sol	low, medium, high, xhigh, max, ultra	Not exposed
gpt-5.6-terra	low, medium, high, xhigh, max, ultra	Not exposed
gpt-5.6-luna	low, medium, high, xhigh, max	Not exposed

Appendix: sources and exact revisions

Source	Exact revision	License / rights
codex-auto-model-router	`01a327701f28740f106e9782c7f14bc195f861b7`	MIT
ui-skills	`59193a75432e5f41c6980f53b9e6a7bd56919947`	MIT
gstack	`4d48d39e5119311b4cff905db75f97aaa8cb35ce`	MIT
awesome-gamedev-agent-skills	`184b72be02884f782a0058f2eeae2ce4277453e4`	Apache-2.0
meowa-skills	`288e9127e13a5f2293db529dbee5f79aedaeb371`	MIT
codex-game-studio	`f3c50a0552f562eafc8178973dc48c29cd0d50fa`	MIT
blender-mcp	`dbbf836ad4b1025f14a2b3b504c43903f39e0b04`	GPL-3.0-or-later
game-assets-installed	`Separately installed skill`	MIT (see meowa-skills source LICENSE)
superpowers	`2ce8aa15d05bd57213e6f746f953aafb7ded7a8b`	MIT
pixelrag	`cf91bd4fc6891a3530627a5304a3ca566e3160c2`	Apache-2.0
atlas-components	`e99428f025da7d391ccc470ade42e559d202b8df`	No repository-level grant found; review third-party notices and obtain rights before redistribution
agent-beacon	`90b19ae0626fd6fd060c62ddbc70827a23242f4e`	MIT
headroom	`17d38e5699b7d7c69ac6addcd762f39e42dd7f00`	Apache-2.0
threejs	`6060921710839609bce347d4a154a207f757d8f7`	MIT
reverse-skill	`cab634bd855fc287f6e420c1f36fd1a6b9245960`	MIT; preserve dependency notices for redistributed components
rea	`0cd7ef456bf960da265e04b341e82a24b1e377d5`	MIT; preserve dependency notices for redistributed components

Appendix: official references and status limits

threejs-official - read

open3dlab - automated read blocked by Anubis challenge; no assets or licenses verified

nmap - referenced; no installation or live assessment performed

wireshark - referenced; no installation or live assessment performed

kali - referenced; no installation or live assessment performed

burp - referenced; no installation or live assessment performed

ghidra - referenced; no installation or live assessment performed

suricata - referenced; no installation or live assessment performed

metasploit - referenced; no installation or live assessment performed

osint - referenced; no installation or live assessment performed

hashcat - referenced; no installation or live assessment performed

aircrack-ng - referenced; no installation or live assessment performed

zap - referenced; no installation or live assessment performed

splunk - referenced; no installation or live assessment performed

nginx - referenced; no installation or live assessment performed

haproxy - referenced; no installation or live assessment performed

No new external product runtime or security target was executed and no real token-savings percentage was measured. Structural checks, management behavior and reference integrity were exercised. Consult the release evidence for publication and remote revision status; this manual does not guarantee future executions.